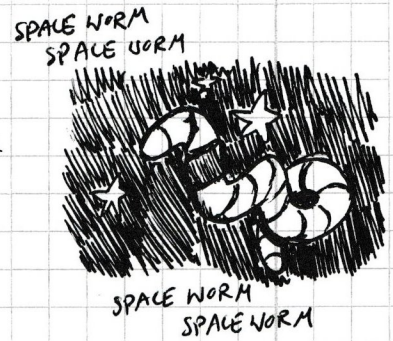


Access Control

grant <Rights>
on <Table>
to <Userlist>
[with grant option]

→ select update delete
→ relation / view name



revoke Rights
on Table
from Userlist
[restrict | cascade]

→ restrict: revokes rights from granted users
→ cascade

Role Model: access via role that is then assigned to user (create role RoleName)

Ch 8: Transactions

> sequence of operations that transfer the database from a consistent state to a (possibly ~~modified~~) consistent state (follows ACID principle)

Semantic integrity: correct (consistent) DB state after transaction

Processing integrity: avoid errors caused by "simultaneous" access to same data by multiple users

Two laws of Concurrency

- program should still run ~~correctly~~ correctly
- should not be ~~significantly slower~~ significantly



Atomicity: either execute entirely or not at all

Consistency: (integrity preservation) DB inconsistent state before and after

Isolation: user should have impression to be only one working on DB

Durability: after successful transaction, result should be "permanently" stored

Serial execution always works (~~Consistency~~) but is slow (Isolation not given)

Mixed execution takes same time (Isolation) but can lead to errors (Consistency)

Allocation of Data Static allocation means a transaction explicitly specifies at design time read/write access

Dynamic allocation objects allocated at runtime

T1	T2	A
read(A)		10
	read(A)	10
write(A)		11
	write(A)	11

Lost Update

write → write dependency

> reads are out of sync, so when the writes are executed (based on previous read, it will lead to ~~inconsistent~~ inconsistency

write $\rightarrow$ read dependency (dirty read) : value was read but not
that read value has not been
permanently stored yet

07/04/26

DB

Contd.

read $\rightarrow$ write (~~unrepeatable~~ unrepeatable read) : value was processed between
reads but first read is then
being worked with?
inconsistent reading

Phantom Problem: does the data exist or not?